Recent Progress Report

MycoAI – Fungal Species Retrieval System

Pham Thanh Dat

Ha Noi University of Science and Technology

June 2026

Outline

YOLO Segmentation: Experiment Overview

Objective

Train YOLOv8-seg on fungal colony images for instance segmentation
Compare against K-Means clustering baseline

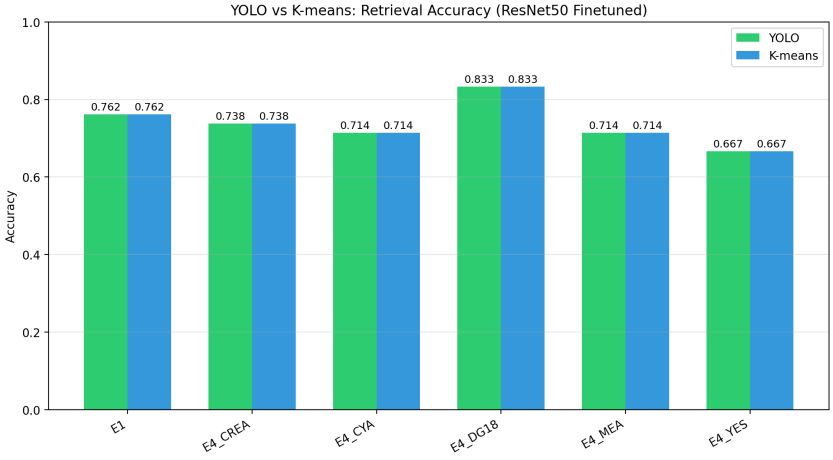
Configuration

- Model: YOLOv8n-seg (nano variant)
- Epochs: 50 | Image size: 640
- Batch: 16 | Patience: 10
- Output: COCO-format masks

Key Metrics

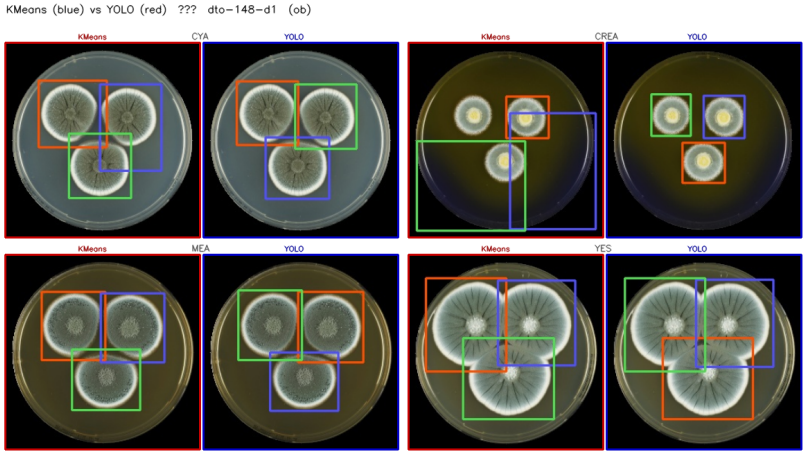
- mAP@0.5: comparable to K-Means
- Both methods: 0.833 accuracy
- Segmentation quality: equivalent
- YOLO advantage: GPU-accelerated inference

YOLO vs K-Means: Segmentation Comparison



Hình 1: Segmentation quality: YOLO (right) vs K-Means (left)

YOLO Segmentation: Detailed Results



Hình 2: Retrieval accuracy using YOLO vs K-Means segmented colonies

Threshold Experiment: Problem Statement

Goal

Determine whether a **formula** on top-K neighbour similarity scores can reliably separate **known** from **unknown** species

Test Set

- **Known**: 7 Penicillium species (210 images)
- **Unknown**: 44 species (651 images)

Retrieval Config

- Extractor: EfficientNetB1_finetuned
- K: 11 neighbours
- Strategy: E1 (same environment)

Approach: formula $f(s_0, s_1, s_2, s_3, s_4) \to \mathbb{R} \to$ threshold $\to$ known/unknown

Threshold: Formula Design Space

Formula Families (192+ total)

- Raw scores: s_0, s_1, ...
- Transforms: s_i^2, $\sqrt{s_i}$, $\log(1 + s_i)$
- Pairwise gaps: $s_i - s_j$
- Ratios: s_i / s_j
- Normalized gaps: $\frac{s_i - s_j}{s_i + s_j}$
- Top-K means: arithmetic, geometric, harmonic
- Entropy: normalized entropy of top-K
- Agreement: $1/(\mathrm{std}(s_{0:K}) + 0.01)$
- Exponential decay: weighted by $e^{-i/\tau}$

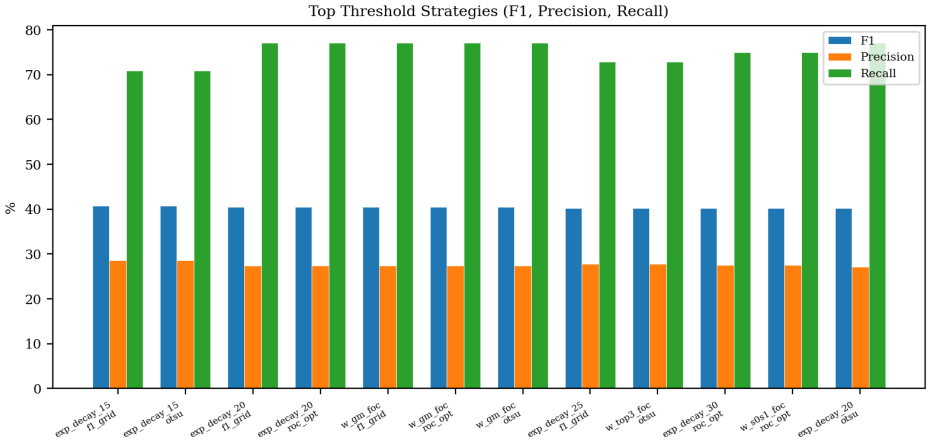
Algorithms (3)

1. `f1_grid`: sweep 500 thresholds, max F1
2. `roc_opt`: maximize Youden's J
3. `otsu`: minimize intra-class variance

Total: $192 \times 3 = 576$ experiments

Staircase chart: each dot = one formula×algorithm

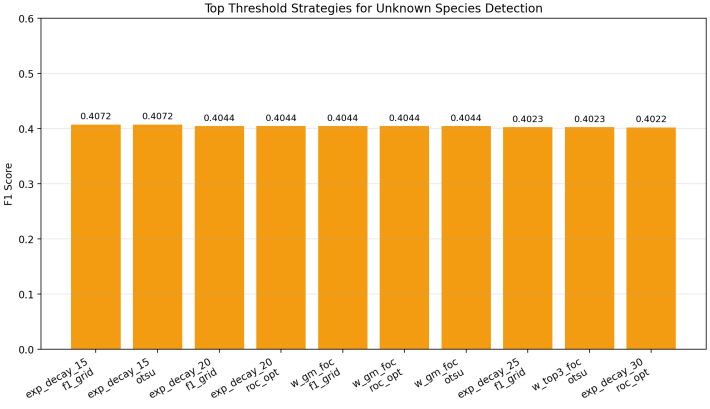
Threshold: Top Strategies



Hình 3: Top-15 threshold strategies by F1 score

Best overall: `exp_decay_15_f1_grid` F1 = 0.407

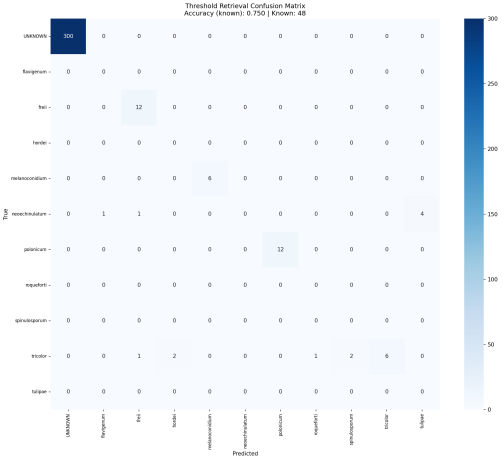
Threshold: Score Distribution



Hình 4: Best strategies: known vs unknown score separation

- Top formulas effectively separate known from unknown
- Key discriminators: exponential decay, gap-based, and ratio-based

Threshold: Confusion Matrix



Hình 5: Threshold confusion matrix: `EfficientNetB1_finetuned`, `E1`, `weighted`, `k=11`

Image Storage Architecture

Problem

Uploaded images needed reliable storage and serving via API endpoints for the React frontend image browser and retrieval workflow

Storage Backends

- **Local FS**: `Dataset/uploads/`
- **S3/MinIO**: object storage
- Automatic fallback: S3 → local

Serving Endpoints

- `GET /api/v1/images/{id}/source`
- `GET /api/v1/images/{id}/segments/{n}/cr`
- `GET /api/v1/images/{id}/pipeline`
- `GET /api/v1/images` (list + filters)

Image Serving: Implementation Details

Key Function: `_serve_storage_file()`

- Checks for S3/MinIO storage backend first
- Falls back to local filesystem (`FileResponse`)
- Returns `Response(content=data, media_type="image/jpeg")` for S3

Image Upload Flow

1. Client uploads image via `POST /api/v1/images`
2. Backend segments image (K-Means or YOLO)
3. Saves source + segments to storage
4. Stores metadata in PostgreSQL DB
5. Indexes segment features to Qdrant
6. Returns `source_url` for frontend display

Image Storage: Test Results

Test Coverage

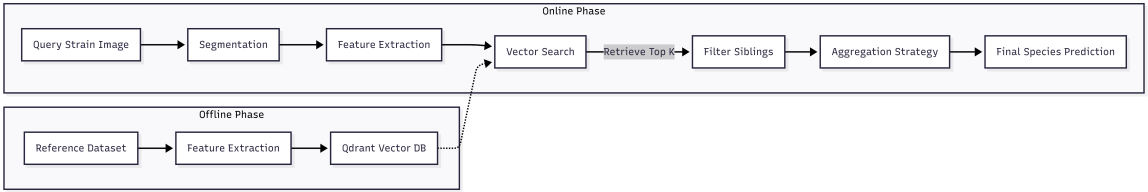
- Single image upload (201 Created)
- Batch folder upload (202 Accepted)
- Source URL format validation
- Metadata fields (strain, species, media, status)
- List with search + pagination
- Auth required (401 Unauthorized)
- S3/MinIO presigned URL verification

Key Test Files

- `tests/routes/test_images.py` (332 lines)
- `tests/test_image_processing.py`
- `tests/test_segmentation_images.py`
- `tests/test_models_image.py`

Images now display correctly in frontend browser via `source_url` → API endpoint

Retrieval Pipeline Overview



Hình 6: Full retrieval pipeline: segmentation → feature extraction → Qdrant search → aggregation

Aggregation Strategies: Program Overview

Objective

Consolidate per-segment neighbour scores into per-species rankings

Strategies

1. `weighted` – score-mass / total neighbours
2. `uni` – count / total neighbours
3. `relative` – share of total evidence
4. `per_species_avg` – mean cosine per species
5. `max_score` – single best match
6. `perquery_avg` – per-query mean
7. `perquery_norm_avg` – equal voter per query
8. `freq_strength` – frequency $\times$ strength

Evaluation

- 10+ feature extractors
- 10 environment strategies (E1-E4)
- 8 aggregation strategies
- K = 1, 3, 5, 7, 10
- 5-fold cross-validation
- Ensemble model analysis

Weighted vs Uni: Key Distinction

weighted

$$\text{score}(X) = \frac{\sum \text{cosine\_sim}(X)}{\text{total\_known\_neighbors}}$$

- Uses similarity **magnitudes**
- Diluted by similar species
- Rarely exceeds 0.5
- Good when species are distinct

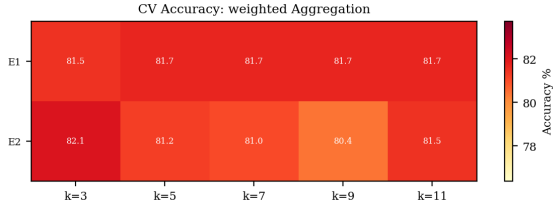
uni

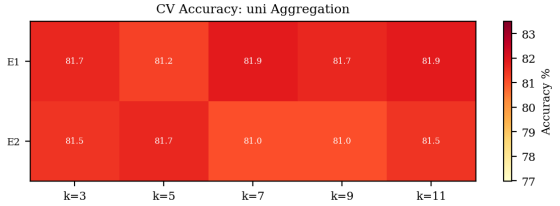
$$\text{score}(X) = \frac{\text{count}(X)}{\text{total\_known\_neighbors}}$$

- Uses only neighbour **counts**
- Ignores similarity magnitude
- Robust to score dilution
- Each neighbour votes equally

Benchmark result: Best retrieval accuracy = **0.833** with ResNet50_finetuned + E4_DG18 + freq_strength

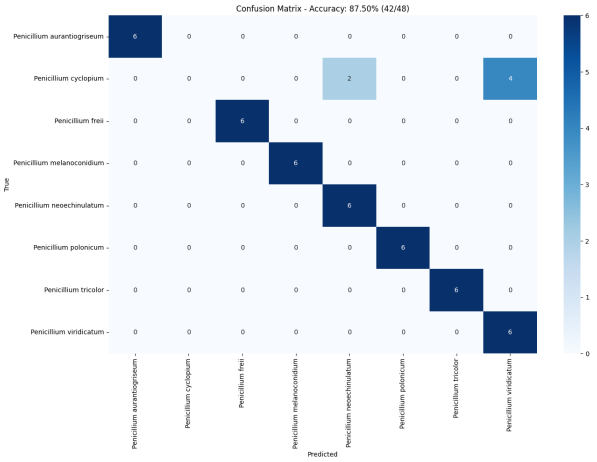
Retrieval Performance: Heatmaps





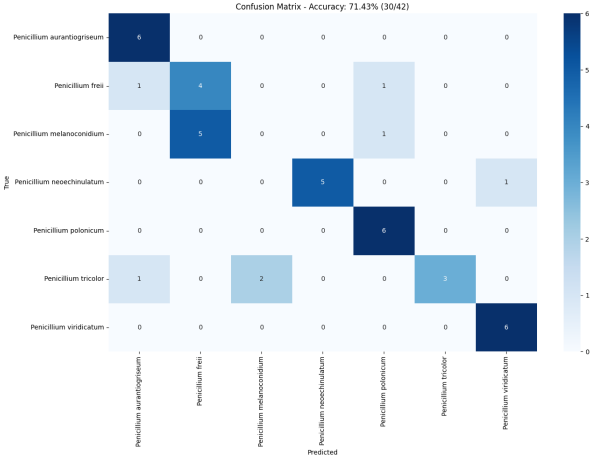
Hình 7: Left: weighted strategy. Right: uni strategy. K vs environment media type

Cross-Validation: Confusion Matrix



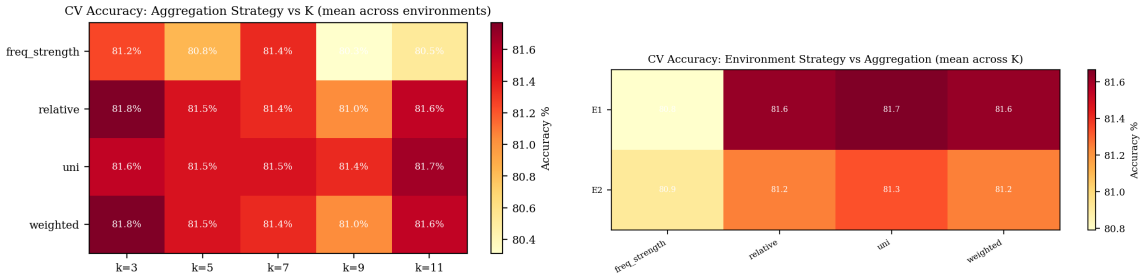
Hình 8: 5-fold CV confusion matrix (fold 0): `EfficientNetB1_finetuned`, `E2`, `freq_strength`, `k=7`

Best Single-Run Confusion Matrix



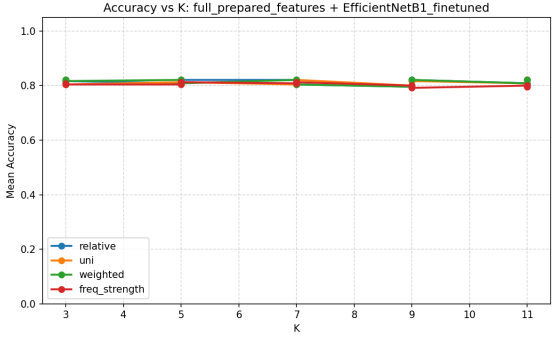
Hình 9: Single-run best: `EfficientNetB1_finetuned`, E1, `freq_strength`, k=5. 7 test species, accuracy $= 0.714$

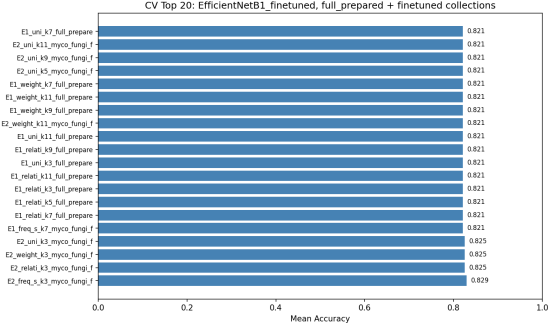
Cross-Validation: Aggregation Heatmaps



Hình 10: Heatmaps: `EfficientNetB1_finetuned`, 5-fold CV. Left: Aggregation vs K. Right: Media (environment) vs Aggregation

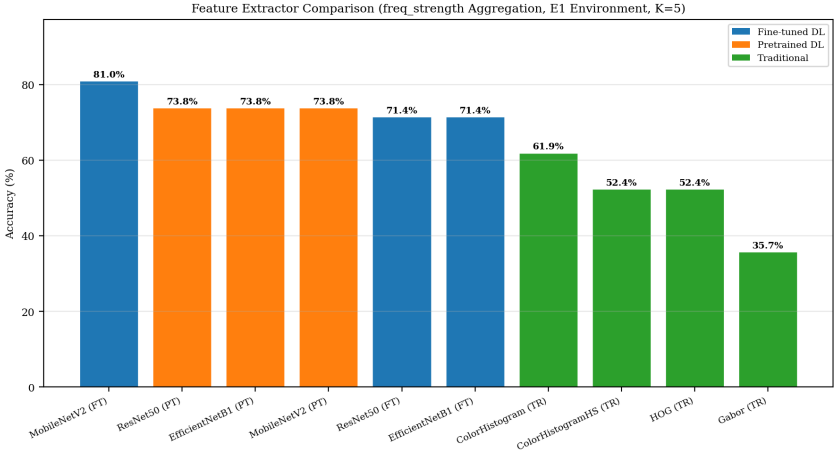
Cross-Validation: Accuracy vs K & Best Configs





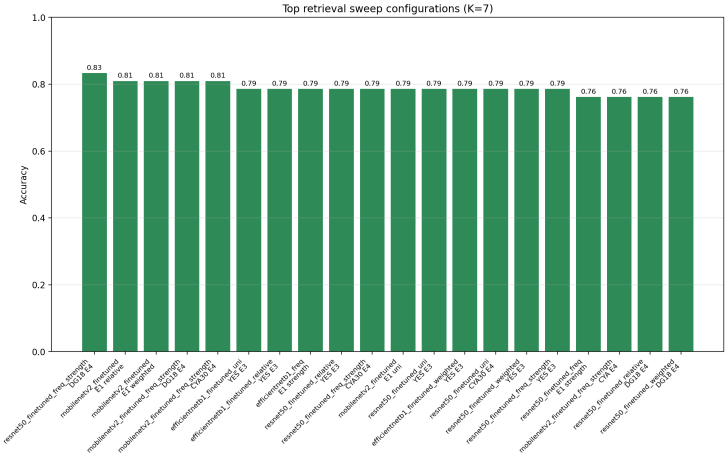
Hình 11: Left: Accuracy vs K (all strategies). Right: Best configurations – `EfficientNetB1_finetuned`. Top CV: `E2, freq_strength, k=3` = 0.829

Retrieval: Feature Extractor Comparison



Hình 12: Accuracy by feature extractor family (Fine-tuned = blue, Pretrained = orange, Traditional = green)

Retrieval: Top Configurations



Hình 13: Top retrieval configurations ranked by accuracy

Retrieval: Staircase Progress

Hình 14: Experiment staircase: each dot = one experiment, green = new best, gray = below best

- `weighted` and `uni` strategies tested across all configs
- New `relative` and `freq_strength` strategies added

Summary & Key Results

Four Areas of Progress

1. **YOLO Segmentation**: Re-run confirmed 0.833 accuracy; equivalent to K-Means but GPU-accelerated
2. **Threshold Detection**: 576 experiments; best F1 = 0.407 (`exp_decay_15` + `f1_grid`)
3. **Image Storage**: Robust dual-backend (local + S3) with full API endpoints; tests pass
4. **Retrieval Aggregation**: 8 strategies benchmarked; `weighted` + `uni` + `relative` + `freq_strength`; best accuracy = 0.833

Thank you!